

Cpp_Engine 학습 가이드

이 문서는 Cpp_Engine에 실제로 적용된 기술들을 현재 코드 그대로 설명합니다.

“왜 이렇게 만들었는지”에 집중합니다.

목차

1. Application / GameLoop 패턴

핵심 아이디어

게임/엔진은 반드시 이 흐름을 갖습니다:

```
초기화 → [입력 처리 → 업데이트 → 렌더링] 반복 → 종료
```

이걸 코드로 표현한 게 **Application**과 **GameLoop**입니다.

실제 코드 — Application.h

```
// src/app/Application.h

class Application {
public:
    Application(int width, int height, const char* title);
    void Run(); // GameLoop를 시작하는 진입점
    IWindow& GetWindow() { return *m_window; }

protected:
    virtual void OnInit() {} // 씬 구성은 여기서
    virtual void OnUpdate(float) {} // 매 프레임 업데이트
    virtual void OnFixedUpdate() {} // 고정 타임스텝 업데이트 (물리용)
    virtual void OnRender(); // 기본 구현 있음

private:
    std::unique_ptr<IWindow> m_window; // 플랫폼 창 (MacWindow 등)
};
```

실제 코드 — GameLoop.cpp

```
// src/app/GameLoop.cpp

void GameLoop::Run(Application& app) {
    Time::Init();
```

```

app.OnInit(); // 처음 한 번만

auto& window = app.GetWindow();

float fixedAcc = 0.0f; // FixedUpdate 누적 시간

while (window.IsOpen()) {
    window.PollEvents(); // 키보드/마우스 이벤트 수집

    Time::Update();

    const float dt = Time::DeltaTime();

    fixedAcc += dt;

    // FixedUpdate: 1/60초 단위로 정확하게 실행 (물리 시뮬레이션용)
    while (fixedAcc >= kFixedStep) {
        app.OnFixedUpdate();

        fixedAcc -= kFixedStep;
    }

    app.OnUpdate(dt); // 매 프레임 (렌더링용 업데이트)
    app.OnRender();

    InputManager::Get().EndFrame(); // 키 JustPressed 상태 초기화
}
}

```

DemoApp 사용 예

```

// src/app/DemoApp.cpp

DemoApp::DemoApp(int width, int height, const char* title)
: Application(width, height, title)
, m_renderer(width, height)
, m_world(m_scene.GetRegistry()) {} // World에 Registry 참조 전달

void DemoApp::OnInit() {
    // 시스템 등록 (처리 순서 = 등록 순서)

    m_world.add_system<InputSystem>();
    m_world.add_system<ScriptSystem>();
    m_world.add_system<TransformSystem>();
    m_world.add_system<CameraSystem>(m_cameraController, m_scene);
    m_world.add_system<RenderSystem>(m_renderer, m_scene, GetWindow());
}

void DemoApp::OnUpdate(float dt) {

```

```
m_world.update(dt); // 등록된 모든 System을 순서대로 실행
}
```

FixedUpdate란?

물리 시뮬레이션은 프레임마다 dt가 달라지면 결과가 달라집니다.

누적기(accumulator) 패턴으로 1/60초 고정 간격을 보장합니다.

```
프레임 dt = 0.02s (50fps) → fixedAcc = 0.02 → FixedUpdate 1회
프레임 dt = 0.05s (20fps) → fixedAcc = 0.05 → FixedUpdate 3회
```

2. ECS — Entity Component System

왜 ECS인가?

전통적인 상속 기반 방식의 문제:

```
GameObject
├── MovableObject
│   ├── Character
│   │   ├── Player
│   │   └── Enemy
│   └── Vehicle
```

- 계층이 깊어질수록 수정이 어려움
- "날아다니는 캐릭터"를 만들려면 다중 상속 필요

ECS 방식:

```
Entity (그냥 숫자 ID)
+ Transform → 위치/회전/스케일
+ MeshRenderer → 메시/재질
+ ScriptComponent → 행동 로직

// 날아다니는 캐릭터? FlyComponent 추가하면 끝
```

세 가지 개념

개념	설명	우리 코드
Entity	고유 숫자 ID	<code>`using Entity = uint32_t`</code>
Component	순수 데이터 구조체	<code>`Transform`, `MeshRenderer`</code>
System	특정 컴포넌트를 가진 Entity 처리	<code>`RenderSystem`, `ScriptSystem`</code>

실제 코드로 보면

```
// Entity 생성 (숫자 하나를 받는 것)
Entity player = m_scene.CreateEntity();

// 컴포넌트 부착
auto& reg = m_scene.GetRegistry();
reg.add<Transform>(player, Transform{});
reg.add<MeshRenderer>(player, MeshRenderer{mesh, material});
reg.add<ScriptComponent>(player, {std::make_shared<RotatorScript>(40.f)});

// System 등록 — variadic template으로 생성자 인자 전달
m_world.add_system<RenderSystem>(m_renderer, m_scene, GetWindow());
```

System이 동작하는 방식

```
// src/renderer/gl/GLRenderer.cpp — OpaquePass 내부
for (auto {transform, meshRenderer} : reg.view<Transform, MeshRenderer>()) {
    if (!meshRenderer.visible || !meshRenderer.mesh) continue;

    const Mat4 model = transform.GetWorldMatrix(reg);
    // ... 렌더링
}
```

`view<A, B>()`는 A와 B 컴포넌트를 둘 다 가진 Entity만 순회합니다.

3. Sparse Set — ECS의 핵심 자료구조

왜 Sparse Set인가?

`std::map<Entity, Component>` 사용 시 → 메모리가 흩어져 캐시 미스 발생, 느림

`Component array[MAX_ENTITIES]` 사용 시 → Entity가 없는 구간도 메모리 차지, 낭비

Sparse Set은 두 배열을 조합합니다:

```
Entity 3, 7, 15에 컴포넌트가 있을 때:

sparse[3] = 0 sparse[7] = 1 sparse[15] = 2 (나머지는 SIZE_MAX = "없음")
dense[] = {3, 7, 15}
components = {c3, c7, c15} ← 빈틈 없이 연속! 캐시 친화적
```

순회는 `components[]`를 처음부터 끝까지 읽으면 끝 — 메모리가 연속되어 빠릅니다.

실제 코드 — ComponentPool.hpp

```
// src/ecs/ComponentPool.hpp
template<typename T>
```

```

class ComponentPool {
public:
void add(Entity entity, T component) {
assert(!has(entity) && "Entity already has this component");

if (entity >= sparse.size())
sparse.resize(entity + 1, SIZE_MAX); // SIZE_MAX = "없음" 표시

sparse[entity] = dense.size(); // sparse에 dense 인덱스 기록
dense.push_back(entity); // dense에 entity ID 추가
components.push_back(std::move(component)); // 실제 데이터 추가 (이동, 복사 없음)
}

void remove(Entity entity) {
size_t idx = sparse[entity]; // 제거할 슬롯
Entity last = dense.back(); // 마지막 슬롯의 Entity

// O(1) 삭제: 제거할 자리에 마지막 요소를 덮어쓰고 마지막을 pop
dense[idx] = last;
components[idx] = std::move(components.back());
sparse[last] = idx; // 마지막 Entity의 sparse 인덱스 업데이트

dense.pop_back();
components.pop_back();

sparse[entity] = SIZE_MAX; // 제거한 Entity는 "없음"으로
}

T& get(Entity entity) {
return components[sparse[entity]]; // sparse로 인덱스 찾아서 반환
}

bool has(Entity entity) const {
return entity < sparse.size() && sparse[entity] != SIZE_MAX;
}

private:
std::vector<T> components; // 실제 컴포넌트 데이터 (연속 메모리)
std::vector<Entity> dense; // dense 인덱스 → entity ID
std::vector<size_t> sparse; // entity ID → dense 인덱스
};

```

4. Transform 계층 구조와 dirty 패턴

계층 구조란?

부모-자식 관계. 부모가 움직이면 자식도 따라 움직입니다.

```
부모 오브젝트 (pos: 0,0,0)
└── 자식 오브젝트 (localPos: 0,1,0)

월드 위치 = 부모WorldMatrix × 자식LocalMatrix

부모가 (5,0,0)으로 이동하면 자식은 (5,1,0)
```

실제 구조체 — Transform.h

```
// src/scene/Transform.h

struct Transform {

    Vec3 localPos = { 0.0f, 0.0f, 0.0f };

    Quat localRot; // 쿼터니언 회전 (기본값 = 단위 회전)

    Vec3 localScale = { 1.0f, 1.0f, 1.0f };

    Entity parent = INVALID_ENTITY; // 부모 Entity ID

    std::vector<Entity> children; // 자식 Entity ID 목록

    mutable Mat4 worldMatrix; // 캐시된 월드 행렬

    mutable bool dirty = true; // 재계산 필요 여부

};
```

TRS 행렬 계산 — Transform.cpp

```
// src/scene/Transform.cpp

Mat4 Transform::GetLocalMatrix() const {

    // Translation × Rotation × Scale 순서로 곱함

    return Mat4::Translate(localPos.x, localPos.y, localPos.z)

        * localRot.ToMat4()

        * Mat4::Scale(localScale.x, localScale.y, localScale.z);

}
```

dirty 패턴이란?

매 프레임 모든 오브젝트의 월드 행렬을 재계산하면 낭비입니다.

“변경된 것만 재계산”하는 게 dirty 패턴입니다.

```
// src/scene/Transform.cpp

Mat4 Transform::GetWorldMatrix(const Registry& reg) const {

    if (dirty) {
```

```

const Mat4 localMatrix = GetLocalMatrix();

Registry& mutableReg = const_cast<Registry&>(reg);

// 부모가 있으면 부모 월드행렬 뒤에 로컬 행렬을 붙임
if (parent != INVALID_ENTITY && mutableReg.has<Transform>(parent)) {
    worldMatrix = mutableReg.get<Transform>(parent).GetWorldMatrix(reg) * localMatrix;
} else {
    worldMatrix = localMatrix;
}

dirty = false; // 계산 완료 → 다음 프레임엔 캐시 반환
}

return worldMatrix;
}

void Transform::MarkDirty(Registry& reg) const {
    dirty = true;

    // 자식 트리 전체를 재귀적으로 dirty 표시
    for (Entity child : children) {
        if (reg.has<Transform>(child)) {
            reg.get<Transform>(child).MarkDirty(reg);
        }
    }
}

```

`SetLocalPos()`를 호출하는 순간 `MarkDirty()`가 자식 트리 전체에 전파됩니다.

실제 계산은 `GetWorldMatrix()`가 처음 호출될 때만 일어납니다 — 지연 계산(lazy evaluation).

부모-자식 연결 방법

```

// 자식 Transform의 SetParent() 호출
childTf.SetParent(childEntity, parentEntity, registry);

// → 내부에서 parent.children에 child 추가, MarkDirty 전파

```

5. 쿼터니언 (Quaternion)

왜 Euler 각도가 문제인가?

Euler 각도(pitch/yaw/roll)는 짐벌락(Gimbal Lock) 문제가 있습니다.

카메라를 위아래로 90도 돌리면 두 축이 겹쳐서 좌우 회전이 이상해지는 현상입니다.

실제 코드 — Quat.h

```
// src/math/Quat.h

struct Quat {
    float x = 0.0f, y = 0.0f, z = 0.0f, w = 1.0f;

    // 축-각도에서 쿼터니언 생성
    static Quat FromAxisAngle(Vec3 axis, float angle) {
        axis = axis.normalized();
        const float halfAngle = angle * 0.5f;
        const float s = std::sin(halfAngle);
        return { axis.x * s, axis.y * s, axis.z * s, std::cos(halfAngle) };
    }

    // 오일러 각도 → 쿼터니언 (YZX 순서: yaw → pitch → roll)
    static Quat FromEuler(float pitchDeg, float yawDeg, float rollDeg) {
        constexpr float degToRad = 3.14159265f / 180.0f;
        const Quat yaw = FromAxisAngle({ 0,1,0 }, yawDeg * degToRad);
        const Quat pitch = FromAxisAngle({ 1,0,0 }, pitchDeg * degToRad);
        const Quat roll = FromAxisAngle({ 0,0,1 }, rollDeg * degToRad);
        return (roll * pitch * yaw).normalized();
    }

    // 두 회전 합성 — 해밀턴 곱
    Quat operator*(const Quat& o) const {
        return {
            w * o.x + x * o.w + y * o.z - z * o.y,
            w * o.y - x * o.z + y * o.w + z * o.x,
            w * o.z + x * o.y - y * o.x + z * o.w,
            w * o.w - x * o.x - y * o.y - z * o.z
        };
    }

    // 렌더링에 쓰는 4x4 회전 행렬로 변환
    Mat4 ToMat4() const;

    // 역회전
    Quat conjugate() const { return { -x, -y, -z, w }; }

    // 길이 1로 정규화
    Quat normalized() const;
```



```
};
```

실제 사용 예 — RotatorScript

```
// src/script/RotatorScript.h

void OnUpdate(Entity self, Registry& reg, float dt) override {
    auto& tf = reg.get<Transform>(self);

    const float angle = m_degreesPerSec * dt * (3.14159265f / 180.0f);

    // 현재 회전에 Y축 회전을 누적
    tf.SetLocalRot(tf.localRot * Quat::FromAxisAngle({0,1,0}, angle), reg);
}
```

침벌락이 없고 보간이 자연스럽습니다. CameraController에서 pitch를 ± 89 도로 클램프하는 이유도 수직 방향 특이점을 피하기 위함입니다.

6. OpenGL 렌더링 파이프라인

CPU vs GPU

소프트웨어 래스터라이저 (이전 방식):

- CPU가 삼각형을 픽셀로 변환 — 화면을 채울수록 선형으로 느려짐

OpenGL (현재 방식):

- CPU는 “이 메시를 이 행렬로 그려라”는 명령만 전달
- GPU의 수천 코어가 픽셀을 병렬 처리 — zoom을 아무리 당겨도 빠름

GPU에 데이터 올리기 — GLMesh.cpp

```
// src/renderer/gl/GLMesh.cpp

void GLMesh::Upload(const Mesh& mesh) {
    glGenVertexArrays(1, &vao);
    glGenBuffers(1, &vbo);
    glGenBuffers(1, &ebo);

    glBindVertexArray(vao);

    // CPU 메시 데이터를 GPU 버퍼로 전송
    glBufferData(GL_ARRAY_BUFFER,
        mesh.vertices.size() * sizeof(MeshVertex),
        mesh.vertices.data(), GL_STATIC_DRAW);
    glBufferData(GL_ELEMENT_ARRAY_BUFFER,
        mesh.indices.size() * sizeof(int),
        mesh.indices.data(), GL_STATIC_DRAW);
}
```

```

// 정점 레이아웃 정의 (GPU가 데이터를 어떻게 읽을지)
// location 0 = pos(vec3), 1 = uv(vec2), 2 = normal(vec3), 3 = tangent(vec3)

glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, sizeof(MeshVertex),
(void*)offsetof(MeshVertex, pos));

glVertexAttribPointer(1, 2, GL_FLOAT, GL_FALSE, sizeof(MeshVertex),
(void*)offsetof(MeshVertex, uv));

glVertexAttribPointer(2, 3, GL_FLOAT, GL_FALSE, sizeof(MeshVertex),
(void*)offsetof(MeshVertex, normal));

glVertexAttribPointer(3, 3, GL_FLOAT, GL_FALSE, sizeof(MeshVertex),
(void*)offsetof(MeshVertex, tangent));

}

```

버텍스 셰이더 — phong.vert

```

// assets/shaders/phong.vert

#version 330 core

layout (location = 0) in vec3 aPos;
layout (location = 1) in vec2 aUV;
layout (location = 2) in vec3 aNormal;
layout (location = 3) in vec3 aTangent;

uniform mat4 uModel; // 오브젝트 → 월드 변환
uniform mat4 uView; // 월드 → 카메라 변환
uniform mat4 uProjection; // 카메라 → 클립 공간 변환
uniform mat4 uLightMVP; // 광원 시점 MVP (Shadow Map용)

out vec3 vWorldPos;
out vec2 vUV;
out vec3 vNormal;
out vec3 vTangent;
out vec4 vLightClipPos;

void main() {
vec4 worldPos = uModel * vec4(aPos, 1.0);

mat3 normalMat = mat3(uModel);

vWorldPos = worldPos.xyz;
vUV = aUV;
vNormal = normalize(normalMat * aNormal);
vTangent = normalize(normalMat * aTangent);
}

```

```

vLightClipPos = uLightMVP * vec4(aPos, 1.0);

gl_Position = uProjection * uView * worldPos; // MVP 변환
}

```

MVP 행렬 = Model $\times$ View $\times$ Projection — 오브젝트를 화면 좌표로 변환하는 핵심입니다.

Phong 셰이딩

```

최종 색상 = Ambient + Diffuse + Specular

Ambient = 전체적으로 균일하게 밝히는 베이스 조명
Diffuse = 광원 방향과 법선 각도로 결정되는 직사광
Specular = 카메라 방향과 반사각으로 결정되는 하이라이트

```

Retina 대응 — GLRenderer.cpp

```

// src/renderer/gl/GLRenderer.cpp

void GLRenderer::Render(Registry& reg, const Camera& cam, const Light& light, IWindow& window) {

// 논리 픽셀이 아닌 실제 물리 픽셀 크기로 뷰포트 설정

m_width = window.PixelWidth(); // MacWindow: convertRectToBacking 결과

m_height = window.PixelHeight(); // Retina에서 논리 크기의 2배

// ...

}

```

7. Shadow Map

아이디어

광원 시점에서 씬을 한 번 렌더링해서 "깊이 텍스처"를 만듭니다.
본 렌더링 때 각 픽셀이 그 깊이보다 멀면 → 그림자, 가까우면 → 밝음.

실제 코드 — GLRenderer.cpp

```

// src/renderer/gl/GLRenderer.cpp

void GLRenderer::ShadowPass(Registry& reg, const Light& light) {

// 광원 시점의 VP 행렬 생성

m_lightVP = Mat4::Perspective(DegToRad(90.0f), 1.0f, 0.1f, 20.0f)
* Mat4::LookAt(light.position, {0,0,0}, {0,1,0});

// 1024 × 1024 depth FBO에 씬의 깊이값만 기록

glViewport(0, 0, m_shadowSize, m_shadowSize);

glBindFramebuffer(GL_FRAMEBUFFER, m_shadowFBO);

```

```

glClear(GL_DEPTH_BUFFER_BIT);

m_shadowShader.Use();

for (auto [transform, meshRenderer] : reg.view<Transform, MeshRenderer>()) {
    if (!meshRenderer.visible || !meshRenderer.mesh) continue;
    const Mat4 model = transform.GetWorldMatrix(reg);
    m_shadowShader.SetMat4("uLightMVP", m_lightVP * model);
    GetOrUploadMesh(meshRenderer.mesh).Draw();
}

glBindFramebuffer(GL_FRAMEBUFFER, 0);
}

```

ShadowPass → OpaquePass 순서로 두 번 렌더링합니다.

더미 텍스처 — albedo/normalMap 없을 때

```

// albedo가 없으면 1×1 흰색 텍스처를 slot 0에 바인딩 (GPU 경고 방지)
uint8_t white(4) = {255, 255, 255, 255};

glGenTextures(1, &m_whiteTex);

glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA8, 1, 1, 0, GL_RGBA, GL_UNSIGNED_BYTE, white);

// normalMap이 없으면 평면 법선(0.5, 0.5, 1.0) 더미 텍스처를 slot 1에 바인딩
uint8_t flatN(4) = {128, 128, 255, 255};

glGenTextures(1, &m_flatNormalTex);

glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA8, 1, 1, 0, GL_RGBA, GL_UNSIGNED_BYTE, flatN);

```

8. 이벤트 버스 (pub/sub 패턴)

왜 이벤트 버스인가?

System끼리 직접 참조하면 결합도가 높아집니다.

```

// 나쁜 방법: InputSystem이 CameraSystem을 직접 알아야 함
m_cameraSystem->ToggleMode();

```

EventBus를 쓰면 발신자가 수신자를 몰라도 됩니다.

```

// 좋은 방법: 타입으로 이벤트를 발행/구독
EventBus::Emit(CameraModeToggleEvent{});

```

실제 코드 — EventBus.h

```

// src/event/EventBus.h

```

```

class EventBus {
public:
    static EventBus& Get();

    template<typename TEvent>
    static void Subscribe(std::function<void(const TEvent&)> handler) {
        auto& bus = Get();
        auto key = std::type_index(typeid(TEvent)); // 타입을 키로 사용
        bus.m_handlers[key].push_back(
            [handler](const std::any& e) {
                handler(std::any_cast<const TEvent&>(e));
            }
        );
    }

    template<typename TEvent>
    static void Emit(const TEvent& event) {
        auto& bus = Get();
        auto key = std::type_index(typeid(TEvent));
        auto it = bus.m_handlers.find(key);
        if (it == bus.m_handlers.end()) return;
        for (auto& handler : it->second)
            handler(std::make_any<TEvent>(event));
    }

private:
    std::unordered_map<
        std::type_index,
        std::vector<std::function<void(const std::any&)>>
    > m_handlers;
};

```

실제 사용 예

```

// src/systems/CameraSystem.cpp — 구독 (생성자에서 한 번)
EventBus::Subscribe<CameraModeToggleEvent>([this](const CameraModeToggleEvent&) {
    m_controller.ToggleMode();
});

// src/systems/InputSystem.cpp — 발행 (Tab 키 감지 시)

```

```
if (InputManager::Get().IsKeyJustPressed(KeyCode::Tab)) {  
    EventBus::Emit(CameraModeToggleEvent{});  
}
```

9. 스크립트 컴포넌트 패턴

Unity의 MonoBehaviour와 동일한 개념

오브젝트마다 다른 행동을 붙일 수 있는 구조입니다.

실제 코드 — IScript.h

```
// src/script/IScript.h  
  
class IScript {  
  
public:  
  
    virtual void OnInit(Entity self, Registry& reg) {} // 초기화 (선택)  
  
    virtual void OnUpdate(Entity self, Registry& reg, float dt) = 0; // 매 프레임 (필수)  
  
    virtual ~IScript() = default;  
  
};
```

실제 사용 예 — RotatorScript

```
// src/script/RotatorScript.h (header-only)  
  
class RotatorScript : public IScript {  
  
    float m_degreesPerSec;  
  
public:  
  
    explicit RotatorScript(float dps) : m_degreesPerSec(dps) {}  
  
    void OnUpdate(Entity self, Registry& reg, float dt) override {  
        auto& tf = reg.get<Transform>(self);  
  
        const float angle = m_degreesPerSec * dt * (3.14159265f / 180.0f);  
  
        tf.SetLocalRot(tf.localRot * Quat::FromAxisAngle({0,1,0}, angle), reg);  
    }  
  
};  
  
// Entity에 부착  
reg.add<ScriptComponent>(e, {std::make_shared<RotatorScript>(40.f)});
```

ScriptSystem이 실행하는 방식

```
// ScriptSystem::update() 내부
```

```
for (auto& [entity, script] : reg.view<ScriptComponent>()) {
    script.script->OnUpdate(entity, reg, dt);
}
```

10. AssetManager — 캐싱 패턴

문제

같은 OBJ 파일을 두 Entity에 붙이면 두 번 로드하게 됩니다 — 메모리 낭비.

실제 코드 — AssetManager.h / .cpp

```
// src/resource/AssetManager.h

class AssetManager {
public:
    static AssetManager& Get();

    const Mesh* LoadMesh(const std::string& path);
    const Texture* LoadTexture(const std::string& path);

private:
    std::unordered_map<std::string, Mesh> m_meshes; // 경로 → Mesh
    std::unordered_map<std::string, Texture> m_textures; // 경로 → Texture
};
```

```
// src/resource/AssetManager.cpp

const Mesh* AssetManager::LoadMesh(const std::string& path) {
    auto it = m_meshes.find(path);
    if (it != m_meshes.end()) return &it->second; // 이미 로드됨 → 캐시 반환

    Mesh mesh;
    if (!ObjLoader::Load(path, mesh)) return nullptr; // 실패 → 캐싱하지 않음

    auto [inserted, _] = m_meshes.emplace(path, std::move(mesh));
    return &inserted->second; // emplace로 삽입 후 포인터 반환 (안정적)
}
```

`emplace()` 후 포인터를 반환하는 이유: `unordered_map`은 삽입 후 리해시가 발생해도

`emplace()`로 얻은 이터레이터의 `second`는 안정적입니다.

(단, `reserve()`를 호출하거나 다른 삽입이 없어야 포인터가 유지됨)

실제 사용 예

```
// src/app/DemoApp.cpp

const Mesh* mesh = AssetManager::Get().LoadMesh(path); // 처음 → 파일 읽기
const Mesh* same = AssetManager::Get().LoadMesh(path); // 두 번째 → 캐시 반환
// mesh == same (같은 포인터)

// GLRenderer에서도 동일한 캐싱 구조 사용
// CPU Mesh → GPU VAO는 처음 한 번만 업로드
GLMesh& glMesh = GetOrUploadMesh(meshRenderer.mesh);
```